Ders 12 Genişletilmiş Notu: Karakter Problemleri: Rakamlardan Sayı, Büyük Harfe Çevirme ve Palindrom

Ders 12 Genişletilmiş Notu: Karakter Problemleri: Rakamlardan Sayı, Büyük Harfe Çevirme ve Palindrom

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Rakam Karakterlerinden Sayı Oluşturma Problemi
- 2. Birinci Çözüm: İç İçe İki Döngü
- 3. İkinci Çözüm: Tek Döngü ve Sağdan Sola Gezinme
- 4. Küçük Harfli Kelimeyi Büyük Harfe Dönüştürme
- 5. Akış Diyagramından C Diline Geçiş
- 6. Palindrom Kelime Problemi
- 7. Palindromun Birinci Çözümü: `palindrom` Değişkeni
- 8. Palindromun İkinci Çözümü: İki Uçtan Yaklaşma

1. Rakam Karakterlerinden Sayı Oluşturma Problemi

Ders, karakter dizileri (string) ve karakter problemleri konusunun devamıdır. Hoca, bir önceki dersin sonunda bıraktığı soruyla başladı: Rakamlardan oluşan bir karakter dizisi veriliyor. Bu rakamların birlikte oluşturduğu sayıyı tek bir tam sayı değişkeninde saklayan algoritma isteniyor. Örnekte dizide `'3'`, `'0'` ve `'5'` karakterleri var ve sonuç 305 olmalı.

Çözüme geçmeden önce kelimenin uzunluğu N kullanıcıdan alınıyor, sonra bir döngüyle `rakamlar` isimli karakter dizisine okuma yapılıyor. Bu iki adım, bu dersteki bütün sorularda ortaktır.

1.1 Karakter Sabiti ve Karakter Değişkeni

Hoca, diziye okunan değerleri tahtaya `'3'`, `'0'`, `'5'` diye yazarken bir kuralı tekrar etti. Bunlar karakter sabitleri olduğu için tek tırnakla gösterilir. Karakter saklayan değişkenler ise hiçbir zaman tek tırnakla yazılmaz.

> **Hocanın vurgusu:** Tek tırnak yalnızca sabitler içindir. Değişkenlerle hiçbir zaman tek tırnak kullanılmaz.

1.2 Uzunluk Neden Bu Derse Özel Okunuyor?

Hoca, kelime uzunluğunu okumanın bugüne özel olduğunu söyledi. Karakter problemlerini ilk kez gördüğümüz için uzunluk ayrıca alınıyor. C'ye geçtiğimizde uzunluk almayacağız. Çünkü okuma işlemi tamamlandığında, kaç karakter okunduysa onun sonuna sistem otomatik olarak null karakterini koyar. Null karakterinin değeri sıfırdır ve her karakter grubu okumadan sonra sıfırla bitecek şekilde düzenlenir.

2. Birinci Çözüm: İç İçe İki Döngü

Bir öğrencinin önerdiği ilk çözüm, her rakamın basamak değerini iç döngüyle baştan hesaplıyor. Hoca çözümü öğrencinin söylediği gibi akış diyagramına çizdirdi. Düz metin olarak akış şöyledir:

```
1. N okunur.
2. I <- 1 || 1 || N döngüsünde rakamlar[I] okunur.
3. toplam = 0
4. I <- 1 || 1 || N döngüsü:
   4.1 carpan = 1
   4.2 J <- 1 || 1 || N-I döngüsü:
       carpan = carpan * 10
   4.3 toplam = toplam + (rakamlar[I] - '0') * carpan
5. toplam yazdırılır.
```

Dördüncü adımdaki parantez önemlidir. Hoca "hangi grupla çarpıyorum, parantez nerede olacak?" diye sordu. Çıkarma işlemi önce yapılır, çarpan bu farkla çarpılır. Böylece I. rakamın basamak değeri 10^{N-I} ile çarpılmış olur.

Bir öğrenci çözümü görünce "döngü içindeki döngüyü anlamadım, çarpan biraz yanlış oldu" dedi. Hoca buna, bir çözümün yanlış olup olmadığına ancak analiz yaparak karar verilebileceğini söyleyerek cevap verdi ve değişkenleri tabloya yazarak çözümü adım adım izledi.

> **Hocanın vurgusu:** Yanlış olup olmadığını ancak analiz yaparak anlayabiliriz. "Bence çalışmaz" demek yetmez.

2.1 Analiz: `'3' '0' '5'`, N = 3

Analiz sırasında tabloya iç döngünün bitişi önce $N-1$ olarak geçmişti. İkinci turda "yine 2 defa gideceğiz" denirken öğrenci bunun $N - I$ olması gerektiğini söyledi ve hoca tabloyu düzeltti. Sınır $N - 1$ kalsaydı iç döngü her turda 2 kez dönerdi. $N - I$ olunca dönüş sayısı her turda bir azalır.

Düzeltilmiş çözümün izlenmesi şöyledir:

I	İç döngü bitişi ($N - I$)	İç döngü dönüşü	Son `carpan`	`rakamlar[I] - '0'`	`toplam`
1	2	2	100	3	300
2	1	1	10	0	300
3	0	0	1	5	305

- **I = 1:** `carpan` 1 ile başlar. J = 1 için $1 \leq 2$ doğru, `carpan` 10 olur. J = 2 için $2 \leq 2$ doğru, `carpan` 100 olur. Sonra $3 \times 100 = 300$ bulunur ve `toplam` 300 olur.
- **I = 2:** `carpan` yeniden 1 olur. $N - I = 1$ olduğu için iç döngü bir kez döner ve `carpan` 10 olur. `'0' - '0'` sıfır verdiği için `toplam` 300 kalır.
- **I = 3:** `carpan` yeniden 1 olur. J'nin bitişi 0 olduğu için iç döngüye hiç girilmez. $5 \times 1 = 5$ eklenir ve `toplam` 305 olur.

Sonuç 305 olduğu için analiz, öğrencinin algoritmasının doğru çalıştığını gösterdi.

2.2 Rakam Karakterinin Sayısal Değeri

Çözümün kalbi `rakamlar[I] - '0'` ifadesidir. Dizide `'3'` karakteri varsa, `'3'`'ün ASCII karşılığından `'0'`'ın ASCII karşılığı çıkarılır ve sonuç her zaman 3 olur. ASCII tablosundaki değerleri bilmeye gerek yoktur, çünkü iki rakam karakteri arasındaki fark rakamların farkı kadardır.

Soru: Çarpan 100 bulunduktan sonra `toplam` 300'e nasıl ulaştı? **Cevap:** `rakamlar[I]` o anda `'3'` karakteridir. Sıfır sabitinden çıkarılınca 3 değeri elde edilir. $3 \times 100 = 300$ olur. `toplam` 0 olduğu için üzerine eklenince 300 çıkar.

Soru: `'0'` karakteri yerine sayısal 48 değerini çıkarabilir miyiz? **Cevap:** Yanlış değil, yapabilirsiniz, ama hoca bunu istemiyor. Çünkü bu, ASCII tablosundaki değerleri ezberlemek zorunda

olmanız demektir. Sıfırın değerine ihtiyacınız varsa tek tırnak içinde sıfır yazarsınız, zaten değerini kullanmış olursunuz.

> **Hocanın vurgusu:** Küçük a, küçük z, büyük A, büyük Z ile çalışırken de bu sabitleri kullanın. Değerlerini kullanmayın.
>
> **Ek bilgi (derste anlatılmadı):** Karakter farkıyla çalışma yöntemi, karakterlerin tabloda ardışık dizilmiş olmasına dayanır. ASCII tablosunda `'0'-'9'`, `'a'-'z'` ve `'A'-'Z'` grupları kendi içlerinde ardışıktır. Hocanın "değeri bilmenize gerek yok" demesinin dayanağı büyük olasılıkla bu düzendir.

3. İkinci Çözüm: Tek Döngü ve Sağdan Sola Gezinme

Bir öğrenci iki döngüyü tek döngüye indirip indiremeyeceğimizi sordu ve hoca "deneyelim" dedi. Öğrenci döngüyü N'den başlatıp bire kadar birer azaltmayı önerdi. Hoca yalnızca döngü kısmını yeniden yazdı, `toplam = 0` ilk değeri ve okuma adımları önceki çözümle aynı kaldı.

3.1 Atama Yönü

Öğrenci döngünün içini kurarken `rakamlar[I] - '0' = sayi` yazdırdı, yani `sayi` değişkeni sağ tarafta kaldı. Hoca burada durdu.

> **Hocanın vurgusu:** Bu, asla yapmamanız gereken bir yazım şeklidir. Böyle bir ifade gördüğümde otomatik olarak sorunun üzerini çizerim. Atama hiçbir zaman soldan sağa yapılmaz ve bir işlemin üzerine atama yapılamaz.

Doğrusu `sayi = rakamlar[I] - '0'` şeklindedir. Hoca, içimizde bunu zaman zaman yapanlar olduğundan emin olduğunu, bu yüzden hatanın konuşulmasının iyi olduğunu söyledi.

3.2 Çarpanın Yeri

Öğrenci döngünün devamını kuramayınca soru sınıfa açıldı. Bir öğrenci basamak değeri için üs almak üzere yeni bir döngü açmayı önerdi. Hoca buna "hayır" dedi. Başka bir öğrenci doğru yolu gösterdi: `carpan = 1` değerini döngüye girmeden önce vermek ve döngü içinde her turda 10 ile çarpmak. Ortaya çıkan akış şöyledir:

```
1. N okunur.
2. I <- 1 || 1 || N döngüsünde rakamlar[I] okunur.
3. toplam = 0, carpan = 1
4. I <- N || -1 || 1 döngüsü:
   4.1 sayi = rakamlar[I] - '0'
   4.2 toplam = toplam + sayi * carpan
   4.3 carpan = carpan * 10
5. toplam yazdırılır.
```

Soru: Bunun için birden N'e gitmemiz gerekmiyor mu? Çarpan giderek büyüyor. **Cevap:** Buna analiz yaparak karar vereceksiniz.

Soru: Bu haliyle döngüye girebilir mi? N birden büyük olabilir. **Cevap:** Döngü zaten N'den başlıyor, birer azalıyor ve bire büyük eşit olduğu sürece devam ediyor. Döngüler büyükten küçüğe de küçükten büyüğe de kurulabilir. Yeter ki artım ya da azaltım doğru verilsin. Burada büyükten başladığımız için eksi diyoruz, artırsaydık olmazdı.

Soru: I'yı birden başlatıp N'e kadar götürmek daha doğru olmaz mı? **Cevap:** Öyle de yapabilirsiniz, herkes dilediğini yazabilir. Soru, bu yazımın çalışıp çalışmadığıdır. Burada 3'ten 1'e doğru geriye gidiyoruz. Hoca, birden N'e giden bir versiyonun da olduğunu ama yeni soruya geçmek için üzerinde durmayacağını söyledi.

3.3 Analiz: `'3' '0' '5'`, N = 3

Bazı öğrenciler analiz yapmadan çözümün çalışmayacağını düşündü. Birler ile yüzler basamağının yer değiştireceği de söylendi. Başka bir öğrenci ise 5'ten başlanacağını izleyerek çözümün çalıştığını buldu. Hoca değişkenleri birlikte izledi:

carpan	I	rakamlar[I]	sayi	toplam
1	3	'5'	5	5
10	2	'0'	0	5
100	1	'3'	3	305

- **I = 3:** $3 \geq 1$ doğru. `'5' - '0'` ile `sayi` 5 olur. 5×1 eklenir ve `toplam` 5 olur. `carpan` 10 olur.
- **I = 2:** $2 \geq 1$ doğru. Hoca burada önce `sayi`'yı yanlışlıkla 5 olarak okudu, sonra düzeltti. `'0' - '0'` ile `sayi` 0 olur. Sayı 0 olduğu için çarpanın kaç olduğunun önemi yoktur ve `toplam` 5 kalır. `carpan` 100 olur.
- **I = 1:** $1 \geq 1$ doğru. `'3' - '0'` ile `sayi` 3 olur. $3 \times 100 = 300$ eklenir ve `toplam` 305 olur.

Döngüden çıkılır ve 305 yazdırılır. Demek ki bu yazım da doğru çalışır.

3.4 Analiz Tablosu Nasıl Yazılmalı?

Bir öğrenci, akışı satır satır izlediğinde turların sonunda çarpanın 10, 100, 1000 olduğunu, tabloda ise 1, 10, 100 yazıldığını söyleyerek tabloyu neye göre yazmamız gerektiğini sordu.

Hoca şöyle açıkladı: İçeri girildiği anda `carpan` 1'dir, işlem yapıldıktan sonra 10 olur. Tablodaki farklılık yalnızca bir gösterim meselesidir ve çok önemli değildir. Daha düzgün yazmak istersek değerleri değiştikleri yerden sonra bir adım aşağıya kaydırmak gerekir. Örneğin `carpan` 1, I, `rakamlar[I]` ve `sayi` bir satırda yazılır, `toplam` ve yeni `carpan` bir alt satıra geçer. Bu çok uzun bir yazım olacağı için hoca derste değerleri aynı satırda gösteriyor. İki yöntem de mümkündür.

3.5 Hangi Çözüm Daha Verimli?

Hoca ikinci çözümde döngü sayısının azaldığını söyledi. Her turda çarpan 10 ile çarpılarak 1, 10, 100 değerleri elde edilir. Birinci çözümdeki gibi her rakam için çarpmayı yeniden yapmaya gerek kalmaz. İşlem sayısı daha az olduğu için ikinci algoritma daha verimlidir.

> **Ders sonrası not:** `'3' '0' '5'` örneğinde 10 ile çarpma sayısı iki çözümde de 3'tür: birincide $2+1+0$, ikincide her turda 1. Genel olarak birinci çözümde $N(N-1)/2$, ikincide N çarpma yapılır. Hocanın "işlem sayısı daha az" sözü büyük olasılıkla rakam sayısı arttıkça geçerli olan genel durum içindir. İkinci çözümde son turda bulunan 1000 değeri de sonuçta kullanılmaz.

4. Küçük Harfli Kelimeyi Büyük Harfe Dönüştürme

Hoca yeni soruyu Algoritma 12 olarak yazdırdı: "Küçük harflerden oluşan bir kelimeyi büyük harflere dönüştüren algoritmanın akış diyagramını çiziniz." Bunu bir uppercase fonksiyonu gibi düşünebiliriz. Size `abc` verilecek, siz bunu `ABC`'ye dönüştüreceksiniz. Soru çözülmeden önce gelen sorular şunlardı:

Soru: Çıktı da verecek miyiz, yoksa yalnızca değiştirecek miyiz? **Cevap:** Çıktı da vereceksiniz.

Soru: Kelimenin kaç harften oluştuğunu başta alalım mı? **Cevap:** Bu örnekte de alabilirsiniz.

Soru: Bir önceki soruda `sayi` diye değişken kullanmak yerine `rakamlar[I] = rakamlar[I] - '0'` desek olur mu? **Cevap:** Hayır. Karakter dizisine sayı değeri atarsanız karakterin kendisini kaybetmiş olursunuz. Ama böyle bir engel yoksa teknik olarak yapabilirsiniz.

Soru: Çıktıyı diziye bir yazı olarak mı vereceğiz? **Cevap:** For döngüsü içinde elde ettiğiniz yeni değeri karakter karakter ekrana yazarsınız.

4.1 Çözüm

Bir öğrencinin çözümü üç döngüden oluşuyor. Hoca, vakit kazanmak için kullanıcı mesajlarını çizmediğini, bizim yazmayı unutmamamız gerektiğini söyledi.

```
1. N okunur.
2. I <- 1 || 1 || N döngüsünde kelime[I] okunur.
3. I <- 1 || 1 || N döngüsü:
   buyuk[I] = kelime[I] + 'A' - 'a'
4. I <- 1 || 1 || N döngüsünde buyuk[I] yazdırılır.
```

4.2 Neden Çalışır?

Hoca mantığı şöyle özetledi: Oradaki karakterin küçük `'a'`'ya uzaklığı neyse, büyük karşılığının büyük `'A'`'ya uzaklığı da odur. Örneğin küçük `'b'`, `'a'`'dan 1 uzaklıktadır. Demek ki büyük karşılığı da `'A'`'dan 1 uzaklıktadır. Önce harften `'a'` çıkarılır, sonra bu uzaklık `'A'` sabitinin üzerine eklenir:

$$\text{buyuk}[I] = \text{kelime}[I] - \text{'a'} + \text{'A'}$$

> **Ek bilgi (derste anlatılmadı):** Bu formül yalnızca küçük harfler için kurulmuştur, soru da girdinin küçük harflerden oluştuğunu varsayar. Türkçeye özgü `ç`, `ğ`, `ı`, `ö`, `ş`, `ü` harfleri temel ASCII tablosunda yoktur. Bu harflerde formül büyük olasılıkla doğru sonuç vermez.

4.3 Sonuç Neden Harf Olarak Görünür?

Bir öğrenci takıldığı yeri şöyle anlattı: Büyük A'dan küçük a'yı çıkarınca sayı elde ediyoruz, kelimeyle toplayınca yine sayı elde ediyoruz. Çıktı harf yerine sayı olmaz mı?

Hoca cevabına temel bir bilgiyle başladı: Bellekte saklanan her bilgi sayısaldır. Karakter, resim ya da ses bilgisi saklasanız da hepsinin bir sayısal karşılığı vardır. `a`, `b`, `c` karakterleri de bellekte ASCII tablosunda denk geldikleri sayı olarak saklanır. C'de karakter olarak sakladığınızı belirten bir tanımlama yaparsınız, ama bütün bu işlemlerin sonunda sayısal değerler toplanıp çıkarılır. Elde edilen değerin isterseniz sayısal karşılığını, isterseniz harf karşılığını ekrana yazdırabilirsiniz. Akış diyagramında bunu belirtmek pek mümkün olmadığı için sayısal değer yazdırılıyormuş gibi görünür. Programda karakter tipinde bir dizi tanımlayıp karakter formatıyla yazdırdığınızda ekranda `A`, `B`, `C` görürsünüz.

4.4 Çözüm Üzerine Sorular

Soru: Bunu tek for döngüsünde yapsak daha iyi olmaz mı? **Cevap:** Okuduğunuz yerde işlemleri yapabilirsiniz, bir sorun yok. Derste daha açık olması için okuma, işlem ve yazdırma döngülerini ayırıyoruz.

Soru: Diziyi çıktılarken yalnızca Output'a ismini yazmak yetmiyor, for döngüsü içinde yazdırmamız gerekiyor, değil mi? **Cevap:** Akış diyagramı gösteriminde tek tek yazdırmamız gerekir. C'de isterseniz tek tek karakter yazdırabilirsiniz, isterseniz `%s` denen özel yazdırma formatıyla karakter grubunun tamamını yazdırabilirsiniz.

Soru: `buyuk[I]` yerine sonucu `kelime[I]`'nın üzerine atasak sorun olur mu? **Cevap:** Küçük halini kaybetmiş oluruz. Ama soru büyük harfe dönüştürmeyi istediği için, küçük haline ihtiyaç duymuyorsak kelimenin üzerine atayabiliriz, hiçbir sorun olmaz.

Soru: Büyük harfleri diziye atamadan aynı döngüde hemen yazdırsak olur mu? **Cevap:** Olur. Ama genelde yeni bir bilgi elde ettiğimizde ya aynı dizi üzerinde değişiklik yaparız ya da yeni bir dizide saklarız, sonra ekrana yazdırırız. Elde edildiği yerde yazdırmayı çok uygulamıyoruz.

Hoca bölümü, başlangıç ve bitiş sabitlerinin önemine dikkat çekerek kapattı. Biraz önce rakamlarla çalışırken `'0'` sabitini, burada harflerle çalışırken `'A'` ve `'a'` sabitlerini kullandık. Karakterlerle çalışırken bu sabitlerden faydalanacağımız yerler olacak. Birçok editörde bulunan uppercase fonksiyonunun çalışma prensibini de görmüş olduk.

5. Akış Diyagramından C Diline Geçiş

Hoca palindrom sorusuna geçmeden iki noktaya değindi. Birincisi, soruların sınıfta tartışılmasının nedeniydi. Bu yöntem biraz uzuyor gibi görünse de farklı bakış açıları, yanlış da olsa,

tartışılmış oluyor ve doğruyla yanlış arasındaki fark öğreniliyor. Aktif olarak fikir yürütülen bir konu daha çabuk öğrenilir.

İkincisi, bugüne kadar algoritmaları akış diyagramlarıyla ifade ettik. Bundan sonra C programlama anlatılacak ve bunun için gereken konular hızlıca geçilecek: değişken tanımlama, atama, toplama gibi işlemler, mod alma, hangi söz diziminin (syntax) kullanıldığı, `if`, `for` ve `while` yapılarının nasıl ifade edildiği. Bu dersten sonra şimdiye kadar çizilen algoritmik çözümlerin kodu yazılabilecek.

6. Palindrom Kelime Problemi

Yeni soru şudur: "Verilen bir kelimenin palindrom kelime olup olmadığını bulan algoritmanın akış diyagramını çiziniz."

Palindrom, tersten okunduğunda ve düzden okunduğunda aynı olan kelimedir. Örneğin `ses` ve `kabak`. Bu kelimeler genelde tek sayıda harfe sahip olsa da çift sayıda harften oluşan örnekler de vardır. Hoca anlamlı olmayan bir örnek de ekledi: `baab`. Kelimenin karakter sayısı N ve kelimenin kendisi dışarıdan okunuyor.

6.1 Sözel Fikir

Hoca akış diyagramına geçmeden önce bu işi kağıt üzerinde nasıl yapacağımızı sözel olarak tarif etmemizi istedi. Bir öğrenci, yarıya kadar olan harfleri diğer yarıdakilerle karşılaştırmayı ve birinci harfi sondan birinci harfle eşitliğine bakmayı önerdi. Bunun için iç içe iki for döngüsü kullanılabileceğini de söyledi.

Hoca, iç içe iki for döngüsünün karmaşıklığı çok artırdığını söyledi. Onun yerine, döngünün her adımında soldan bir karakter ve sağdan bir karakter alıp eşit olup olmadıklarına bakılabilir. Her adımda soldaki indis bir artırılır, sağdaki indis bir azaltılır. İki ayrı döngüye gerek yoktur.

6.2 For mu, While mı?

Hoca bu sorunun net bir cevabı olduğunu söyledi. Kelimenin uzunluğuna ve benzerliğin bozulduğu yere bağlı olarak hangi noktada çıkacağımızı bilmiyoruz. Bu yüzden `while` kullanılır.

Soru: For döngüsüyle çözüp yanlış olduğunda bir sayacı artırsam çözüm sayılır mı? **Cevap:** Muhtemelen doğru çalışır ama efektif bir çözüm değildir. `while` ile ilk hatanın görüldüğü yerde çıkılabilir, oysa `for` ile sonuna kadar bakmak gerekir. `abcd`'de a ile d farklı olduğu için geri kalanlara bakmaya gerek yoktur. Buradaki örnekler kısa. Kelime uzadıkça gereksiz kontrol sayısı daha da artar.

7. Palindromun Birinci Çözümü: `palindrom` Değişkeni

Bir öğrenci önce palindrom olduğunu varsaydığını, yani `palindrom = 1` kabul ettiğini söyledi. Koşul bozulursa bu değişken sıfır yapılacak ve döngü devam etmeyecek. Yarıya kadar gitmek için de $I < N \text{ DIV } 2 + 1$ koşulunu kurdu.

7.1 True Kolu Boş Olamaz

Öğrencinin çözümündeki if kutusu `kelime[I] = kelime[N-I+1]` diye soruyordu: eşitse `palindrom = 1`, değilse `palindrom = 0`. Hoca `kabak` analizine başladığında ilk karşılaştırmada true koluna gidildiğini gösterip eşit olduğunda değişkeni yeniden 1 yapmaya gerek olmadığını söyledi.

Soru: True kolunu işlem kutusu olmadan doğrudan bağlayabilir miydik? **Cevap:** Hayır. True kolu olmayan if olmaz.

Doğru yol koşulu ters kurmaktır. Bir öğrencinin de söylediği gibi "eşit değil mi?" diye sorulur: `kelime[I] ≠ kelime[N-I+1]` ise true kolunda `palindrom = 0` yapılır, false kolunda hiçbir şey yazılmaz. Değişkenin değeri zaten yalnızca eşit olmadığında değişecektir.

> **Hocanın vurgusu:** True kolu olmayan if olmaz. Bir kolda yapılacak iş yoksa koşulu tersine çevirin.

Düzeltilmiş akış şöyledir:

```
1. N okunur.
2. I <- 1 || 1 || N döngüsünde kelime[I] okunur.
3. palindrom = 1, I = 1
4. While I < N DIV 2 + 1 AND palindrom = 1:
   4.1 kelime[I] ≠ kelime[N-I+1] ise palindrom = 0
   4.2 I = I + 1
5. palindrom = 1 ise "palindrom", değilse "palindrom değil" yazdırılır.
```

7.2 Analiz: `kabak`, N = 5

5 DIV $2 = 2$ olduğu için sınır $2 + 1 = 3$'tür.

I	$I < 3$	`palindrom`	`kelime[I]`	$N - I + 1$	`kelime[N-I+1]`	Karşılaştırma
1	doğru	1	`k`	5	`k`	eşit
2	doğru	1	`a`	4	`a`	eşit
3	yanlış	1				döngü biter

Çıkıldığında `palindrom` 1 olduğu için ekrana "palindrom" yazılır. Ortadaki `b` harfine hiç bakılmadığına dikkat edin.

7.3 Analiz: `baab` ve `abcd`

`baab` için N = 4'tür. 4 DIV $2 + 1 = 3$ olduğu için I = 1 ve I = 2 kontrol edilir. `kelime[1]` ile `kelime[4]` ikisi de `b`, `kelime[2]` ile `kelime[3]` ikisi de `a`'dır (ara adım eklendi). I = 3 olunca döngü biter, sonuç "palindrom"dur.

Soru: `baab` analizindeki 4, 1, 1 değerleri nereden geldi? **Cevap:** N 4 olur, I 1 olur, `palindrom` 1 olur. `kelime[1]` ile `kelime[4]` de `b`'dir.

`abcd` için ilk karşılaştırmada `a` ile `d` farklı olduğundan `palindrom` 0 olur. I artar. While'ın ilk koşulu sağlansa da ikinci koşul sağlanmadığı için döngüden çıkılır. Sonuç "palindrom değil"dir.

7.4 Döngüden Sonra Neden If Var?

> **Hocanın vurgusu:** While'dan neden çıktığımızı bilmiyoruz. Gerçekten `palindrom` 0 mı oldu, yoksa kontrol edilecek kısımların hepsi kontrol edildi ve sorun yok mu? Bunu anlamak için döngüden sonra bir if gerekir.

7.5 Yarıya Kadar Gitme Sınırı

Bir öğrenci koşulu `I <= N DIV 2` yaptığını söyledi. Hoca o anda sınır değeri konusunda emin olamadığını belirtti. Dersin sonraki kısmında da N DIV $2+1$ kısmını ayrıntılı analiz etmediğini ve kontrol etmekte fayda olduğunu söyledi.

> **Ders sonrası not:** `I < N DIV 2 + 1` ile `I <= N DIV 2` koşulları tam sayılarda aynı I değerlerini geçirir. Python ile 1-10 harfli kelimeler üzerinde yapılan denemede iki koşul da her kelimede doğru sonuç verdi. Bu yüzden öğrencinin önerdiği yazım büyük olasılıkla doğrudur ve birinci çözümün sınırı da büyük olasılıkla bir hata içermez.

8. Palindromun İkinci Çözümü: İki Uçtan Yaklaşma

Hoca "bunu daha şık bir hale getirebilir miyiz?" diye sordu. Bir öğrenci I'yı artırıp diğer indisi azaltan bir döngü önerdi. Hoca buna benzer bir çözümü kendisi yazdı: `I = 1`, `J = N` diye başlıyor ve bir `while` döngüsünde iki uçtan ortaya doğru ilerliyor. Yazarken bazı hatalar olduğunu baştan söyledi ve bunları tek tek konuşarak düzeltti.

8.1 Gereksiz Değişken ve İlk Düzeltme

Önce `palindrom = 1` ifadesinin artık gerekmediği görüldü, koşulun kendisi while'a yazılınca bu değişkene ihtiyaç kalmıyor. Hoca ilk olarak while koşulunu "I büyük J" olarak yazmıştı. `kabak` için "1, 5'ten büyük mü?" diye sorunca koşulun ters olduğu ortaya çıktı ve `I < J` olarak düzeltildi.

Bir öğrenci, I ile J birbirini geçtikten sonra aynı çiftlerin ikinci kez kontrol edileceği kaygısını dile getirdi. Hoca buna bakacağımızı söyledi. Koşul `I < J` olunca, I ile J buluştuğunda ya da birbirini geçtiğinde döngü biter.

8.2 Döngüden Sonraki If: Sınır Değerleri

Asıl tartışma döngüden sonraki if koşulundaydı. Her düzeltme bir örneği kurtarıp diğerini bozuyordu.

`I < J` denemesi, `kabak`, N = 5: (1, 5) eşit, (2, 4) eşit, sonra I = J = 3 olur ve döngü biter. If'te "$3 < 3$ mü?" sorusunun cevabı hayırdır ve "not palindrome" yazılır. Bu yanlıştır. Hoca koşulu `I <= J` yapınca bu örnek düzeldi.

`I <= J` denemesi, `baab`, N = 4: (1, 4) eşit, (2, 3) eşit, sonra I = 3, J = 2 olur. $3 < 2$ yanlış olduğu için döngü biter. If'te "$3 \leq 2$ mi?" sorusunun cevabı hayırdır ve yine "not palindrome" yazılır. Bu da yanlıştır.

`I >= J` düzeltmesi: Bir öğrenci son if'te "I büyük eşit J" demeyi önerdi. Son akış şöyledir:

```
1. N okunur.
2. I <- 1 || 1 || N döngüsünde kelime[I] okunur.
3. I = 1, J = N
4. While I < J AND kelime[I] = kelime[J]:
   4.1 I = I + 1
   4.2 J = J - 1
5. I >= J ise "palindrome", değilse "not palindrome" yazdırılır.
```

8.3 Son Çözümün Örneklerle Kontrolü

Hoca düzeltilmiş çözümü farklı örneklerle denedi. `baab` ve `kabak` için çalıştığını belirttikten sonra diğer örneklere geçti:

Kelime	N	Eşit bulunan çiftler	Döngü bitişinde (I, J)	$I \geq J$	Sonuç
`kabak`	5	(1, 5), (2, 4)	(3, 3)	doğru	palindrome
`baab`	4	(1, 4), (2, 3)	(3, 2)	doğru	palindrome
`abcd`	4	yok, (1, 4) farklı	(1, 4)	yanlış	not palindrome
`ses`	3	(1, 3)	(2, 2)	doğru	palindrome
`abca`	4	(1, 4); (2, 3) farklı	(2, 3)	yanlış	not palindrome
`abc`	3	yok, (1, 3) farklı	(1, 3)	yanlış	not palindrome

`abca` örneğini çözümün çalışmayacağı bir örnek olarak bir öğrenci önerdi, ama çözüm doğru sonuç verdi. Hoca yine de örneği farklı bir durum olduğu için önemli buldu ve teşekkür etti: (1, 4) eşleşip (2, 3) eşleşmediğinde de çözümün doğru çalıştığını görmemiz gerekiyordu. `abc` için de bir öğrenci ortada bir sorun olabileceğini düşündü. Oysa 1 ve 3 farklı olduğu için döngüden hemen çıkılır, öğrenci de bunu görünce anladığını söyledi.

> **Hocanın vurgusu:** Sınır değerler her zaman sizi yanıltabilir. Bir çözüm iki örnekte doğru çalışır, birinde yanlış çalışır. O zaman sınır değerlerini değiştirmek gerekir.

8.4 İki Çözümün Karşılaştırılması

Hoca iki yaklaşımı şöyle karşılaştırdı:

- **`palindrom` değişkenli çözüm:** Bir değişken 1'e kurulur, 1 olduğu sürece devam edilir, koşul sağlanmazsa değiştirilir. N DIV $2 + 1$ için fazladan bir işlem yapılır. Döngüde iki kontrol vardır: bir if kontrolü, bir de while kontrolü. Öğrenmenin ilk aşamasında karışıklık olmaması için bu tarz yaklaşımlar daha çok kullanılır ve makul bir çözümdür.
- **İki indisli çözüm:** İçeride if yoktur, koşul doğrudan while'a konur. Kelimeye soldan ve sağdan yaklaşılır, yalnızca I artırılır ve J azaltılır. Sınır değerleri çok dikkatli seçilmelidir. Oturması biraz zaman alabilir, ama zaman içinde bu çözüme doğru kaymanız beklenir.

Soru: `palindrom` değişkenli çözüm yalnızca daha az efektif mi, yoksa kaçırdığı durumlar da var mı? **Cevap:** Kaçırdığımız bir durum olmaması lazım, ama kontrol etmenizde fayda var. Sınır kısmı 7.5'te ele alındı.

Dersin Toparlaması

Bu derste karakter dizileriyle çalışmanın temel fikri üç soruda görüldü. Bellekteki her bilgi sayısal olduğu için karakterlerle toplama ve çıkarma yapılabilir ve bu işlemlerde ASCII değerleri yerine `'0'`, `'a'`, `'A'` gibi sabitler kullanılır. Rakamlardan sayı oluşturmada bu fikir `rakamlar[I] - '0'` ile, büyük harfe çevirmede `kelime[I] + 'A' - 'a'` ile kullanıldı. Palindromda ise ilk farklılığın nerede çıkacağı belli olmadığı için `while`, indis olarak da iki uçtan yaklaşma kullanıldı. Bütün soruların ortak dersi, bir çözümün doğruluğuna analizle karar verilmesi ve tek-çift uzunluklu, eşleşen ve eşleşmeyen örneklerin ayrı ayrı denenmesidir.

Sırada C programlama var. Hoca, akış diyagramıyla çözülen algoritmaların kodunun bundan sonra yazılabileceğini, muhtemelen 8. haftada da matrislere geçileceğini söyledi. Kaydın son bölümü bölüm oryantasyonuna ayrılmıştır ve idari içerik olduğu için bu notta yer almaz.

> **Hocanın vurgusu:** Bu dersi çalışmanın en güvenilir yolu, dersi dinleyip notunu kendiniz almak ve çözülmüş algoritmaları kapatıp kendiniz yeniden çözmeye çalışmaktır.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.